

## Выполнение задания 1

Модуль с проверками ввода вещественного числа в интервале (-1, 1) без 0

```
1 def get_float_input(prompt, validator=None): 2 usages new *
2 C:\Users\germa\453504_ZHEBRIK_10\IGI\LR3\Task1.py
3 Generic function to get a valid float with optional validation.
4 Protects against ValueError and logic errors.
5 """
6 while True:
7     try:
8         val = float(input(prompt))
9         if validator is None or validator(val):
10             return val
11         print("Error: Input value is out of range.")
12     except ValueError:
13         print("Error: Please enter a numeric value (e.g., 0.5).")
14
15 def ask_to_continue(): 1 usage new *
16 """Asks the user if they want to run the program again."""
17 choice = input("\nDo you want to calculate for another x? (y/n): ").lower()
18 return choice == 'y'
```

Декоратор над функцией вычисления суммы и сама функция степенного ряда с дополнительным функционалом - построение таблицы

```
def table_decorator(func):
    def wrapper(*args, **kwargs):
        print("\n" + "=" * 70)
        # ^10 означает выравнивание по центру в поле из 10 символов
        print(f"| {'x':^10} | {'n':^5} | {'F(x)':^12} | {'Math F(x)':^12} | {'eps':^10} |")
        print("-" * 70)

        # Вызов основной функции (calculate_taylor)
        result = func(*args, **kwargs)

        print("=" * 70)
        return result

    return wrapper

@table_decorator
def calculate_taylor(x, eps, max_iter=500):
    n = 0
    current_sum = 0.0
    term = 1.0 # Первый член ряда x^0 = 1

    while n < max_iter:
        current_sum += term
        n += 1

        term *= x

        if abs(term) < eps:
            math_f = 1 / (1 - x)
            print(f"| {x:10.4f} | {n:5} | {current_sum:12.6f} | {math_f:12.6f} | {eps:10.1e} |")
            return True

    print(f"\n Warning: Max iterations ({max_iter}) reached for x={x}.")
    return False
```

## Задание 2

Итератор в Python - объект, который реализует интерфейс с двумя методами `__iter__` (вернуть объект итератора) `__next__` (перейти к следующему элементу, при конце последовательности (StopIteration))

Генератор - функция, которая при вызове не возвращает объект сразу, а возвращает объект генератора, через который можно лениво получать значения в момент запроса (а не хранить все в памяти сразу)

Пример генератора во втором задании

```
import input_check
```

```
def init_with_zero_stop():
    '''
    Generator to initialize sequence until zero dont meet
    '''

    print('Enter number sequence (0 stop sequence)')
    while True:
        val = input_check.get_int_input('>')
        if val == 0:
            return
        yield val
```

Функция для подсчета суммы и максимума

```
def task_2_business_logic():
    sequence = data_init.init_with_zero_stop()

    total_sum = 0
    max_val = None

    for num in sequence:
        total_sum += num
        if max_val is None or num > max_val:
            max_val = num

    return total_sum, max_val
```

Результат работы программы

```
Enter number sequence (0 stop sequence)
>5
>3
>-1
>2
>0
Max element is 5. Sum is 9
```

### Задание 3

```
def count_characters_in_range(text, start_char='g', end_char='o'):
    count = 0
    for char in text:
        if start_char <= char <= end_char:
            count += 1
    return count

def main():
    text = input("Enter text: ")
    print("Count elems > g and < o = ",
count_characters_in_range(text))
main()
```

### Задание 4

```
def get_words_count(text):
    return len(text.split())

def get_letters_frequency(text):
    freq = {}
    for char in text.lower():
        if char.isalpha():
            freq[char] = freq.get(char, 0) + 1
    return freq

def get_alphabetical_phrases(text):
    # Разбиваем по запятой
    parts = text.lower().split(',')
    # Убираем пробелы по краям и пустые строки
    clean_parts = [p.strip() for p in parts if p.strip()]
    # Сортируем (в Python сортировка строк идет по алфавиту)
    return sorted(clean_parts)

def main():
    raw_text = (
        """So she was considering in her own mind, as well as she could,
        for the hot day made her feel very sleepy and stupid,
        whether the pleasure of making a daisy-chain would be worth the
        trouble
        of getting up and picking the daisies, when suddenly a White Rabbit
        with pink eyes ran close by her."""
    )
    print(f"1. Total words: {get_words_count(raw_text)}")

    print("\n2. Letter frequencies:")
    letters = get_letters_frequency(raw_text)
    for char in sorted(letters):
        print(f"'{char}': {letters[char]}", end=" | ")

    print("\n\n3. Phrases in alphabetical order:")
    phrases = get_alphabetical_phrases(raw_text)
    for i, phrase in enumerate(phrases, 1):
        print(f"{i}. {phrase}")

main()
```

## Задание 5

Здесь были создан генератор для вещественных чисел и пользовательский ввод для них

```
def init_float_generator(size, min_val = -100.0, max_val = 100.0):
    for _ in range(size):
        val = random.uniform(min_val, max_val)
        yield val

def init_float_by_user(size, min_val = -100.0, max_val = 100.0):
    inp = []
    print('Enter float number sequence')
    for _ in range(size):
        val = input_check.get_float_input(f"Element [{len(inp) + 1}]:")
        inp.append(val)
    return inp
```

## Сама программа

```
import data_init

def analyze_positive_elements(data):
    """
    Can work with generator and list
    """
    min_pos = None
    first_idx = -1
    last_idx = -1
    total_between_sum = 0

    # Временный список, если нам нужно считать сумму "между"
    # Для генератора нам все равно придется где-то хранить значения,
    # чтобы вычислить сумму после того, как узнаем, где был ПОСЛЕДНИЙ
    элемент.
    items = list(data)

    if not items:
        return None, 0

    #min positive
    positives = [x for x in items if x > 0]
    if positives:
        min_pos = min(positives)

    #Find range to sum
    for i, val in enumerate(items):
        if val > 0:
            if first_idx == -1:
                first_idx = i
            last_idx = i

    #Sum
    if first_idx != -1 and last_idx != -1 and last_idx > first_idx +
1:
        total_between_sum = sum(items[first_idx + 1: last_idx])

    return min_pos, total_between_sum

def main():
    n = int(input('Enter sequence size : '))
```

```
data = data_init.init_float_generator(n)
tup = analyze_positive_elements(data)
print(f"Min pos : {tup[0]}. Between sum : {tup[1]}")

main()
```